

CMakeProject2 - Model Dosyasından Cevap Üretimi

Eğitim son

Bu doküman, eğitimle oluşan model.ckpt dosyasının sunucu tarafında nasıl yüklendiğini ve kullanıcı mesajından Türkçe cevaba nasıl dönüştürüldüğünü açıklar.

Ozet: Eğitim vs Kullanım

- Eğitim: veri okunur, loss hesaplanır, ağırlıklar güncellenir, model.ckpt yazılır.

- Kullanım

Örnek İstek

Flutter uygulaması POST /api/v1/chat ile JSON gönderir:

```
{"prompt": "Merhaba", "max_tokens": 64, "temperature": 0.7}
```

Adım 0 - Sunucu açılınca model yüklenir

serve komutu çalışınca HybridChatBackend oluşur. model.ckpt ve tokenizer_data RAM'e alınır. Checkpoint header ve tensor ağırlıkları okunur; TransformerModel inşa edilir.

Adım 1 - HTTP istegi

ChatHttpServer POST /api/v1/chat JSON gövdesini parse eder.

Adım 2 - Backend seçimi

HybridChatBackend local veya openai backend'e yönlendirir.

Adım 3 - Intent + prompt şablonu

Örnek: Niyet: sohbet. Kullanıcı: Merhaba Asistan:
Model eğitimde 'Asistan:' sonrasını öğrenmişti.

Adım 4 - Metin -> token ID

BpeTokenizer metni sayılara çevirir.

Adım 5 - Prefill

Prompt 124 katmandan geçer; son pozisyondan logits (her token için skor) üretilir.

Adım 6 - Sampling

Temperature, top-k, top-p ile bir sonraki token seçilir.

Adım 7 - Decode döngüsü

Her yeni token için forward + yeni logits; maxTokens veya EOS'a kadar.

Adım 8 - Token -> metin

Yeni tokenlar tokenizer ile Türkçe metne çevrilir.

Adım 9 - Temizle + JSON

Örnek cevap: {"text":"Selam!","intent":"sohbet","error":null}

Görsel Akış

```
JSON -> ParseChatRequest -> Intent + prompt  
-> Encode -> Prefill (128 katman) -> logits  
-> Sample -> Decode döngüsü -> Decode metin  
-> Sanitize -> JSON -> Flutter
```

Tam Kod (Inference Pipeline)

Asagida uctan uca zincirin tam fonksiyonlari var. Transformer katman detayi (transformer.cpp, attention.cpp) ayri - onlar dosyadaki agirliklarla matris carpimi yapar.

1. Model yukleme - chat_session.cpp

```
Status ChatSession::Load(const std::string& modelPath,
                        const std::string& tokenizerPath) {
    auto model = LoadModel(modelPath); // .ckptq -> QuantCheckpoint::LoadRuntime
    if (!model.IsOk()) {
        return Status::Fail(model.GetError().code, model.GetError().message);
    }
    auto tokenizer = BpeTokenizer::Load(tokenizerPath);
    if (!tokenizer.IsOk()) {
        return Status::Fail(tokenizer.GetError().code, tokenizer.GetError().message);
    }
    model_ = std::move(model.Value());
    tokenizer_ = std::move(tokenizer.Value());
    engine_.emplace(*model_);
    generator_.emplace(*engine_, sampling_);
    loaded_ = true;
    return Status::Ok();
}
```

2. Checkpoint okuma - checkpoint.cpp

```
Result<TransformerModel> Checkpoint::Load(const std::string& path) {
    std::ifstream input(path, std::ios::binary);
    CheckpointHeader header{};
    ReadBytes(input, &header, sizeof(header));
    // magic + version dogrula
    const ModelConfig config = HeaderToConfig(header);
    TransformerModel model(config);
    for (std::uint32_t tensorIndex = 0; tensorIndex < header.numTensors; ++tensorIndex) {
        std::string name;
        auto tensor = ReadTensor(input, name);
        AssignTensor(model, name, tensor.Value());
        // embedding, layer.0.attn..., lm_head
    }
    return Result<TransformerModel>::Ok(std::move(model));
}
```

3. Prompt sablonu - dialogue_format.cpp

```
std::string BuildInferencePrompt(const std::string& intent,
                                const std::string& userPrompt,
                                const std::string& systemPrompt) {
    const std::string normalizedIntent =
        intent.empty() ? nlp::DefaultIntent() : nlp::NormalizeIntentLabel(intent);
    std::string sequence = "Niyet: " + normalizedIntent + ".";
    if (!systemPrompt.empty()) {
        sequence += " Sistem: " + systemPrompt;
    }
    sequence += " Kullanici: " + userPrompt + " Asistan:";
    return sequence;
}

std::string SanitizeGeneratedResponse(std::string text) {
    const std::string marker = "Asistan:";
    const std::size_t markerPos = text.find(marker);
    if (markerPos != std::string::npos) {
```

```
    text.erase(0, markerPos + marker.size());
}
const std::string intentMarker = "Niyet:";
const std::size_t intentPos = text.find(intentMarker);
if (intentPos != std::string::npos) {
    text.erase(intentPos, text.size());
}
while (!text.empty() && (text.front() == ' ' || text.front() == '\n'
    || text.front() == '\r')) {
    text.erase(text.begin());
}
return text;
}
```

4. Local backend - local_chat_backend.cpp

```
ChatResponse LocalChatBackend::Complete(const ChatRequest& request) {
    ChatResponse response;
    if (!session_.IsLoaded()) {
        response.error = "local model is not loaded";
        return response;
    }
    if (request.prompt.empty()) {
        response.error = "prompt is empty";
        return response;
    }
    std::lock_guard<std::mutex> lock(mutex_);
    inference::SamplingConfig& sampling = session_.Sampling();
    const float savedTemperature = sampling.temperature;
    const bool savedGreedy = sampling.greedy;
    sampling.temperature = request.temperature;
    sampling.greedy = request.greedy;
    const std::string intent = request.intent.empty()
        ? nlp::ClassifyIntent(request.prompt)
        : nlp::NormalizeIntentLabel(request.intent);
    const std::string modelPrompt =
        training::BuildInferencePrompt(intent, request.prompt);
    const auto result = session_.Complete(modelPrompt, request.maxTokens, rng_);
    sampling.temperature = savedTemperature;
    sampling.greedy = savedGreedy;
    if (!result.IsOk()) {
        response.error = result.GetError().message;
        return response;
    }
    response.text = training::SanitizeGeneratedResponse(result.Value());
    response.intent = intent;
    return response;
}
```

5. Oturum tamamlama - chat_session.cpp

```
Result<std::string> ChatSession::Complete(const std::string& prompt,
    const std::size_t maxNewTokens, std::mt19937& rng) {
    if (!loaded_ || !generator_.has_value()) {
        return Result<std::string>::Fail(ErrorCode::InvalidArgument,
            "chat session is not loaded");
    }
    if (prompt.empty()) {
        return Result<std::string>::Fail(ErrorCode::InvalidArgument,
            "prompt is empty");
    }
    const std::vector<TokenId> promptIds =
        tokenizer_.EncodeWithSpecialTokens(prompt, true, false);
    const auto generated = generator_>Generate(promptIds, maxNewTokens, rng);
    if (!generated.IsOk()) {
        return Result<std::string>::Fail(generated.GetError().code,
            generated.GetError().message);
    }
    const std::vector<TokenId>& allIds = generated.Value();
    if (allIds.size() <= promptIds.size()) {
        return Result<std::string>::Ok(std::string{});
    }
    const std::vector<TokenId> newIds(
        allIds.begin() + static_cast<ptrdiff_t>(promptIds.size()), allIds.end());
    return Result<std::string>::Ok(tokenizer_.Decode(newIds));
}
```

6. Token uretim dongusu - generator.cpp

```
Result<std::vector<TokenId>> TextGenerator::Generate(
    const std::vector<TokenId>& prompt, const std::size_t maxNewTokens,
    std::mt19937& rng) {
    if (prompt.empty()) {
        return Result<std::vector<TokenId>>::Fail(ErrorCode::InvalidArgument,
            "generate requires a non-empty prompt");
    }
    if (maxNewTokens == 0) {
        return Result<std::vector<TokenId>>::Ok(prompt);
    }
    const Dimension vocabSize = engine_.GetModelConfig().vocabSize;
    std::vector<TokenId> sequence = prompt;
    Tensor logits = Tensor::Zeros(Shape{1, vocabSize});
    const Status prefillStatus = engine_.Prefill(prompt, logits);
    if (!prefillStatus.IsOk()) {
        return Result<std::vector<TokenId>>::Fail(prefillStatus.GetError().code,
            prefillStatus.Message());
    }
    for (std::size_t step = 0; step < maxNewTokens; ++step) {
        const TokenId nextToken = Sampler::Sample(logits, config_, rng);
        sequence.push_back(nextToken);
        if (nextToken == config_.eosTokenId) break;
        if (engine_.SequenceLength() >= engine_.GetModelConfig().maxSeqLen) break;
        const Status decodeStatus = engine_.Decode(nextToken, logits);
        if (!decodeStatus.IsOk()) {
            return Result<std::vector<TokenId>>::Fail(decodeStatus.GetError().code,
                decodeStatus.Message());
        }
    }
    return Result<std::vector<TokenId>>::Ok(std::move(sequence));
}
```

7. Forward pass - inference_engine.cpp

```
Status InferenceEngine::Prefill(const std::vector<TokenId>& tokens,
    Tensor& logits) {
    Reset();
    Tensor hidden;
    EmbedTokens(tokens, hidden); // token ID -> vektor
    Tensor output;
    ForwardHidden(hidden, 0, output); // 124 katman
    Tensor lastRow = Tensor::Zeros(Shape{1, model_.GetConfig().hiddenDim});
    const Dimension lastIndex = static_cast<Dimension>(tokens.size() - 1);
    for (Dimension dim = 0; dim < model_.GetConfig().hiddenDim; ++dim) {
        lastRow.At({0, static_cast<Index>(dim)}) =
            output.At({static_cast<Index>(lastIndex), static_cast<Index>(dim)});
    }
    return LogitsFromRow(lastRow, logits); // norm + lm_head -> vocab skorlari
}

Status InferenceEngine::Decode(const TokenId token, Tensor& logits) {
    Tensor hidden;
    EmbedToken(token, hidden);
    const std::size_t cacheStart = cache_.Length();
    Tensor output;
    ForwardHidden(hidden, cacheStart, output);
    return LogitsFromRow(output, logits);
}

Status InferenceEngine::ForwardHidden(const Tensor& hidden,
    const std::size_t cacheStart, Tensor& output) {
    Tensor current = hidden;
    for (std::size_t layer = 0; layer < model_.NumLayers(); ++layer) {
```

```
Tensor next = Tensor::Zeros(Shape{seqLen, model_.GetConfig().hiddenDim});
model_.Layer(layer).ForwardWithCache(current, model_.GetRopeCache(),
                                     cache_, layer, cacheStart, next);

current = std::move(next);
}
output = std::move(current);
return Status::Ok();
}
```

8. Kelime secimi - sampler.cpp

```
TokenId Sampler::Sample(const Tensor& logits, const SamplingConfig& config,
                        std::mt19937& rng) {
    if (config.greedy || config.temperature <= 0.0f) {
        return SampleGreedy(logits); // en yuksek skor
    }
    std::vector<Scalar> row = LogitsRow(logits);
    ApplyTemperature(row, config.temperature);
    ApplyTopK(row, config.topK);
    SoftmaxInPlace(row);
    ApplyTopP(row, config.topP);
    return SampleFromDistribution(row, rng);
}
```

Kisa Ozet

Ne	Nerede
Dosya -> bellek	Checkpoint::Load / QuantCheckpoint::LoadRuntime
Metin -> sayi	BpeTokenizer::Encode
Sayi -> anlam vektoru	EmbedTokens + kayitli agirliklar
Siradaki kelime?	ForwardHidden -> logits
Kelime sec	Sampler::Sample
Sayi -> metin	BpeTokenizer::Decode
HTTP	ChatHttpServer POST /api/v1/chat

Ogrenme egitimde olur; inference sadece kayitli agirliklarla 'bir sonraki token' tahminini tekrarlar. Model bir veritabani degildir; her cevap o anda hesaplanir.